

Pilas y colas en sistemas reales: Undo/Redo de un editor de código y control de tráfico en un firewall

Implementación desde cero y análisis formal de complejidad

Nash Díaz

Estructuras de Datos y Algoritmos

Universidad EAFIT

Agosto de 2026

Resumen

Se implementan desde cero una pila y una cola, cada una en dos representaciones distintas —arreglo dinámico y lista enlazada— expuestas tras una misma interfaz abstracta, y se aplican a dos problemas reales: el mecanismo de deshacer y rehacer de un editor de código y el búfer de recepción con limitador de tasa de un firewall. Se deduce formalmente la complejidad temporal y espacial de cada operación, identificando tres situaciones que requieren análisis amortizado, y se contrastan las cotas obtenidas con mediciones sobre instancias de hasta 10^6 eventos. Los resultados confirman el comportamiento amortizado constante del limitador de tasa, revelan que el costo del dominio puede dominar al de la estructura, y muestran el efecto de la localidad de memoria entre dos representaciones asintóticamente equivalentes.

Palabras clave: pila, cola, tipo abstracto de datos, análisis amortizado, localidad de memoria.

1. Introducción

Este trabajo implementa desde cero dos estructuras de datos unidimensionales y las emplea para resolver dos problemas que aparecen en sistemas reales. No se utilizan `std::stack`, `std::queue`, `std::deque` ni `std::list`: las estructuras se construyen sobre memoria gestionada manualmente y sobre nodos enlazados.

Además de la implementación, el trabajo deduce formalmente la complejidad de cada operación, la contrasta con mediciones experimentales y documenta las decisiones de diseño que el enunciado deja deliberadamente abiertas.

2. Contexto de los dos problemas

Por qué el editor necesita una pila. Cuando un usuario deshace un cambio, el que debe revertirse es el último que aplicó. El orden de reversión es estrictamente inverso al orden de ocurrencia, que es la definición de LIFO. Una cola produciría el comportamiento contrario; un arreglo indexado permitiría el acceso pero no impondría la disciplina, dejando abierta la posibilidad de deshacer en desorden.

Se emplean *dos* pilas y no una porque rehacer necesita recordar lo deshecho. Al deshacer, la operación no se descarta: migra de la pila de deshacer a la de rehacer. Esa migración es lo que permite recorrer el historial en ambos sentidos.

Por qué el firewall necesita una cola. Los paquetes deben procesarse en su orden de llegada, es decir FIFO. El limitador de tasa refuerza la elección: determinar cuántos paquetes llegaron en los últimos T milisegundos exige descartar los más antiguos por el frente mientras entran los recientes por el final. Ese patrón de acceso —salidas por un extremo, entradas por el otro— es exactamente una cola.

3. Fundamentación teórica

3.1. Pila

Colección con disciplina LIFO, con operaciones `push`, `pop`, `top` e `isEmpty`.

Sobre **arreglo dinámico**, los elementos ocupan posiciones contiguas de 0 a $n - 1$ y el tope es siempre la posición $n - 1$. Al llenarse hay dos caminos: declarar la pila llena y rechazar la inserción, o reservar un bloque mayor y trasladar lo existente. Aquí se eligió lo segundo, con lo que la pila nunca se declara llena y el único límite es la memoria disponible.

Sobre **lista enlazada**, cada elemento vive en un nodo con su dato y un enlace al siguiente. La cabeza actúa como tope, de modo que apilar consiste en crear un nodo que apunte a la cabeza anterior. No existe capacidad ni, por tanto, redimensionamiento.

Aplicaciones más allá de este trabajo: la pila de llamadas, la evaluación de expresiones aritméticas, la verificación de paréntesis balanceados, el retroceso en *backtracking* y el recorrido en profundidad de grafos.

3.2. Cola

Colección con disciplina FIFO, con operaciones `enqueue`, `dequeue`, `front`, `isEmpty` e `isFull`.

Sobre **arreglo circular** se mantienen dos índices que avanzan en aritmética módulo la capacidad. Al pasar del final vuelven al inicio, lo que permite reutilizar las posiciones liberadas sin desplazar elementos. Sin esa circularidad, cada `dequeue` obligaría a correr todo el contenido una posición, degradando la operación de $\Theta(1)$ a $\Theta(n)$.

La circularidad introduce una ambigüedad: la condición $head = tail$ se cumple tanto con la cola vacía como con la cola llena, porque los índices dan la vuelta completa. Distinguir las exige información adicional. Las salidas habituales son mantener un contador de elementos, sacrificar una celda —declarando llena cuando $(tail + 1) \bmod C = head$, con lo que solo caben $C - 1$ elementos en C posiciones— o llevar una bandera booleana.

Sobre **lista enlazada** se mantienen punteros al frente y al final. El puntero al final es imprescindible: sin él, encolar exigiría recorrer la lista completa y costaría $\Theta(n)$ en lugar de $\Theta(1)$.

Aplicaciones: planificación de procesos, colas de impresión, búferes de entrada y salida, recorrido en anchura de grafos y control de tráfico en redes.

3.3. Listas enlazadas

Un nodo agrupa un dato y un enlace al siguiente; la cabeza es el punto de entrada y el último nodo enlaza a nulo. Recorrer exige seguir los enlaces uno a uno, de modo que alcanzar el elemento i cuesta $\Theta(i)$. Insertar o eliminar en un extremo cuesta $\Theta(1)$ si se dispone del puntero correspondiente. Buscar un valor cuesta $\Theta(n)$ en el peor caso.

Frente al arreglo las diferencias son tres: acceso indexado en $\Theta(1)$ contra acceso secuencial; redimensionamiento al llenarse contra crecimiento nodo a nodo; y un puntero adicional por elemento, con la consiguiente dispersión de los nodos en el montículo. Esta última diferencia no altera el orden asintótico pero sí el tiempo real, como muestran las mediciones de la Sección 8.

4. Diseño de las soluciones

4.1. El TAD y sus dos implementaciones

Se definieron dos interfaces abstractas, `IStack` e `IQueue`, que fijan *qué* operaciones existen y qué garantiza cada una, sin comprometerse con *cómo* están construidas. Las cuatro clases concretas —`StackArray`, `StackList`, `QueueCircular` y `QueueList`— implementan esas interfaces.

El contrato es uniforme: toda operación que puede fallar devuelve un valor booleano y entrega el resultado por parámetro de salida. Desapilar una pila vacía no es un error fatal sino una operación nula reportada.

Qué permanece igual al cambiar de representación: la interfaz, la semántica de cada operación y el tratamiento de los casos límite. La lógica de ambos problemas está escrita contra punteros a la interfaz y desconoce qué implementación tiene debajo; se elige en tiempo de ejecución.

Qué cambia: el costo puntual del peor caso, el consumo de memoria y la localidad de acceso.

La verificación experimental de esta separación es directa: el programa procesa los siete casos de prueba del Problema 1 con ambas pilas y compara los registros de salida. **Son idénticos en los siete casos.** Una diferencia habría indicado que alguna implementación no respeta el contrato.

4.2. Decisiones de diseño

Crecimiento por duplicación. Al llenarse, la capacidad se multiplica por dos. Con esta política, n inserciones consecutivas provocan traslados que suman $1 + 2 + 4 + \dots < 2n$, de modo que el costo amortizado por inserción es constante. Crecer en una constante k daría un total del orden de $n^2/2k$ traslados, es decir un costo amortizado lineal.

Contador explícito en la cola circular. Se mantiene el número de elementos: vacía equivale a contador cero y llena a contador igual a la capacidad. Se descartó sacrificar una celda porque obligaría a reservar $C+1$ posiciones para admitir C paquetes, y la bandera booleana porque exige actualizarse en cada operación y es más fácil de desincronizar.

Ventana abierta por la izquierda. Una marca expira cuando su tiempo es menor o igual a $t - T$, de modo que la ventana vigente es el intervalo $(t - T, t]$. En consecuencia, un paquete que llega exactamente en $t_0 + T$ ya no cuenta al que llegó en t_0 . La convención contraria es igualmente defendible; lo exigible es elegir una explícitamente y sostenerla.

Orden de comprobación en el firewall. Primero el límite de tasa, después la capacidad del búfer. El limitador es la defensa exterior del sistema: un origen abusivo debe frenarse aunque haya espacio libre.

Solo los paquetes aceptados dejan marca. Un paquete descartado no consumió recursos de procesamiento y no debería penalizar al origen. La postura contraria sería más estricta frente a un ataque de denegación de servicio y también se sostiene.

Posiciones fuera de rango. Se recortan al final del documento en lugar de descartar la operación, para que ninguna edición se pierda en silencio y el programa nunca aborte.

5. Pseudocódigo de las operaciones fundamentales

```
PROCEDIMIENTO push(op)                                // pila sobre arreglo
  SI n = capacidad ENTONCES grow()
  datos[n] <- op
  n <- n + 1

FUNCION pop() -> (exito, valor)
  SI n = 0 ENTONCES RETORNAR (falso, indefinido)
  n <- n - 1
  valor <- datos[n]
  datos[n] <- vacio                                // libera la memoria de la celda
  RETORNAR (verdadero, valor)

PROCEDIMIENTO grow()
  nuevaCapacidad <- capacidad * 2
  nuevo <- reservar bloque de nuevaCapacidad
  PARA i DESDE 0 HASTA n - 1
    nuevo[i] <- datos[i]
  liberar datos
  datos <- nuevo
  capacidad <- nuevaCapacidad

PROCEDIMIENTO push(op)                                // pila sobre lista
  nodo <- nuevo Nodo(op)
  nodo.siguiente <- cabeza
  cabeza <- nodo
  n <- n + 1

FUNCION enqueue(p) -> exito                            // cola circular
  SI n = capacidad ENTONCES RETORNAR falso
  datos[tail] <- p
  tail <- (tail + 1) MOD capacidad
  n <- n + 1
  RETORNAR verdadero

FUNCION dequeue() -> (exito, valor)
  SI n = 0 ENTONCES RETORNAR (falso, indefinido)
  valor <- datos[head]
  head <- (head + 1) MOD capacidad
  n <- n - 1
  RETORNAR (verdadero, valor)

FUNCION purgarExpirados(tActual) -> cantidad
```

```

cantidad <- 0
MIENTRAS ventana no este vacia
  marca <- frente de ventana
  SI marca > tActual - T ENTONCES
    TERMINAR // vigente: las demas tambien
  dequeue(ventana)
  cantidad <- cantidad + 1
RETORNAR cantidad

PROCEDIMIENTO registrarEdicion(op)
  aplicar(op)
  push(pilaUndo, op)
  clear(pilaRedo) // una edicion nueva invalida el rehacer

FUNCION deshacer() -> exito
  (ok, op) <- pop(pilaUndo)
  SI NO ok ENTONCES RETORNAR falso // no-op valido, no error
  revertir(op)
  push(pilaRedo, op)
  RETORNAR verdadero

FUNCION procesarPaquete(p) -> veredicto
  purgarExpirados(p.tiempo)
  SI tamano(ventana) >= L ENTONCES RETORNAR RECHAZO_POR_TASA
  SI bufer esta lleno ENTONCES RETORNAR RECHAZO_POR_BUFER
  enqueue(bufer, p)
  enqueue(ventana, p)
  RETORNAR ACEPTADO

```

6. Análisis de complejidad

6.1. Pila (Problema 1)

Operación	Mejor	Prom.	Peor	Espacial	Justificación
push arreglo, sin redim.	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$	Una escritura y un incremento: número fijo de operaciones elementales, independiente de n
push arreglo, con redim.	$\Omega(1)$	$O(1)^*$	$\Theta(n)$	$\Theta(n)$ tr.	El traslado copia los n elementos; con duplicación, n inserciones suman menos de $2n$ traslados. Durante la copia coexisten dos bloques
push lista	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$	Crear el nodo y reasignar la cabeza; sin recorrido ni capacidad que agotar
pop / top	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$	Acceso directo al extremo activo: índice $n - 1$ en el arreglo, cabeza en la lista
isEmpty	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$	Una comparación
clear (invalidar Redo)	$\Theta(1)$	$O(1)^*$	$\Theta(n)$	$\Theta(1)$	Destruye los n elementos almacenados; amortizado por el argumento de la Sección 6.3

Cuadro 1: Complejidad de las operaciones de pila. * costo amortizado.

6.2. Cola (Problema 2)

6.3. Análisis global

Problema 1. Sobre n eventos, las operaciones de pila aportan $O(n)$ por el argumento amortizado de la duplicación. Sin embargo, **el costo total no es lineal**, y la causa no es la pila sino el documento: insertar o borrar en medio de una secuencia de caracteres cuesta $\Theta(m)$, donde m es la longitud actual del documento. El total queda en $O(n \cdot m)$. Distinguir el costo de la estructura del costo del dominio resulta esencial, y las mediciones lo confirman.

La complejidad espacial es $O(\sum |\text{texto}|)$, no $O(n)$: cada operación almacenada guarda dos cadenas —el contenido nuevo y el previo, necesario para poder revertir— de modo que el espacio depende del volumen de texto y no solo del número de operaciones.

Segundo argumento amortizado. Vaciar la pila de rehacer cuesta $\Theta(|\text{Redo}|)$ en el peor caso puntual. No obstante, cada elemento que llega a esa pila fue antes retirado de la pila de deshacer, y se destruye a lo

Operación	Mejor	Prom.	Peor	Espacial	Justificación
enqueue circular	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$	Escritura, una operación módulo y un incremento. La capacidad es fija: no hay redimensionamiento
dequeue circular	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$	Avance del índice de frente en aritmética modular
Purga de expiradas	$\Omega(1)$	$\Theta(1)^*$	$O(k)$	$\Theta(1)$	Retira k marcas vencidas, $k \leq n$. El corte anticipado es válido porque la cola está ordenada por tiempo: si el frente sigue vigente, el resto también
isFull / isEmpty	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$	Comparación del contador contra 0 o contra la capacidad; el contador es lo que desambigua $head = tail$
enqueue/dequeue lista	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$	El puntero al final evita recorrer para encolar; sin él sería $\Theta(n)$

Cuadro 2: Complejidad de las operaciones de cola. * costo amortizado.

sumo una vez. El trabajo total de vaciado en toda la ejecución está acotado por el número de operaciones de deshacer, que a su vez está acotado por n . El costo **amortizado por edición** es $O(1)$, aunque una edición concreta pueda pagar $\Theta(n)$.

Problema 2. Sobre n paquetes el costo total es $\Theta(n)$. Encolar, desencolar y consultar son $\Theta(1)$, y la purga es $\Theta(1)$ amortizado: una llamada puede retirar hasta k marcas y costar $\Theta(k)$, pero **cada marca se encola exactamente una vez y se desencola a lo sumo una vez** en toda la ejecución. La suma de todas las purgas está acotada por n y, repartida entre n paquetes, da un costo constante por paquete.

La complejidad espacial es $\Theta(C + L)$: el búfer nunca supera su capacidad fija y la ventana nunca retiene más de L marcas. Es independiente de n , lo que significa que el firewall procesa un flujo arbitrariamente largo con memoria acotada.

7. Casos de prueba

Se implementaron los catorce casos exigidos, siete por problema. Cada archivo documenta en su cabecera el resultado esperado. Los siete del Problema 1 se ejecutan además con ambas representaciones de pila.

#	Caso (Problema 1)	Resultado observado
1	Secuencia normal mixta	Documento coherente, sin operaciones nulas
2	Deshacer sin ediciones previas	Tres operaciones nulas reportadas, sin caída
3	Una edición, deshacer, rehacer	Documento de 4 caracteres, Undo= 1, Redo= 0
4	Edición inmediatamente tras deshacer	Un rehacer invalidado; el posterior es nulo
5	50 ediciones y 50 deshacer	Undo= 0, Redo= 50, documento vacío
6	Más rehacer que elementos disponibles	Uno efectivo, tres nulos
7	Crecimiento de capacidad (200 inserciones)	5 redimensionamientos, 248 copias, capacidad final 256
Problema 2		
1	Flujo por debajo de C y L	4 aceptados, 0 rechazos
2	Consulta sobre búfer vacío	Reporte en ceros, sin caída
3	Un único paquete	1 aceptado, ocupación máxima 1
4	Búfer exactamente lleno más uno	4 aceptados, 1 rechazado por capacidad
5	Ráfaga que excede L dentro de T	3 aceptados, 3 rechazados por tasa; el de $t = 2000$ aceptado
6	Desencolar sobre búfer vacío	Primer intento exitoso, siguientes nulos
7	Paquete exactamente en el borde $t_0 + T$	Rechazado, coherente con la ventana abierta

Cuadro 3: Casos de prueba obligatorios y resultados observados.

El caso 7 del Problema 1 merece atención. La teoría predice que 200 inserciones sobre una capacidad inicial de 8 producen redimensionamientos en 8, 16, 32, 64 y 128, es decir cinco eventos y $8 + 16 + 32 + 64 + 128 = 248$ copias acumuladas. **La medición coincide exactamente con la predicción.**

Se verificó adicionalmente con AddressSanitizer la ausencia de fugas de memoria y de comportamiento indefinido, comprobación pertinente dada la gestión manual de memoria.

8. Experimentación

Instancias generadas de forma determinista con semilla 20260830, en los tamaños $n \in \{10^3, 10^4, 10^5, 10^6\}$. Cinco repeticiones por tamaño, medidas con `std::chrono::high_resolution_clock`.

Condiciones: Windows, GCC 13.1.0, indicador -O2, procesador (11th Gen Intel(R) Core(TM) i3-1115G4 @ 3.00GHz).

n	Pila arreglo	Pila lista	Firewall
10^3	4.38 ms (sd 8.51)	0.58 ms (sd 0.03)	0.63 ms (sd 0.03)
10^4	10.93 ms (sd 7.34)	5.92 ms (sd 0.71)	8.87 ms (sd 1.84)
10^5	78.45 ms (sd 20.88)	73.15 ms (sd 10.96)	60.00 ms (sd 3.90)
10^6	1560.12 ms (sd 13.43)	1699.60 ms (sd 202.53)	1055.90 ms (sd 409.97)

Cuadro 4: Tiempo total de ejecución: media y desviación estándar de cinco repeticiones.

n	Pila arreglo	Pila lista	Firewall
10^3	4.38	0.58	0.63
10^4	1.09	0.59	0.89
10^5	0.78	0.73	0.60
10^6	1.56	1.70	1.06

Cuadro 5: Costo por operación en microsegundos. Un comportamiento $O(1)$ se manifiesta como una columna aproximadamente constante.

Como métricas de dominio se registran, en el Problema 1, las operaciones de deshacer y rehacer efectivas frente a las nulas; y en el Problema 2, los aceptados y rechazados por cada causa, la ocupación máxima del búfer, el total de marcas purgadas y la purga máxima puntual.

9. Comparación teórico-experimental

El firewall confirma el costo amortizado constante. Su costo por paquete se mantiene entre 0.60 y 1.06 microsegundos mientras n crece tres órdenes de magnitud. Esa estabilidad es lo que predice el argumento amortizado: aunque una purga concreta pueda retirar muchas marcas, el trabajo total está acotado por el número de marcas encoladas. Si la purga no fuera amortizada, esta columna crecería con n .

El crecimiento del editor es superlineal, y la pila no es responsable. El costo por operación desciende hasta $n = 10^5$ y luego sube a 1.56 y 1.70 microsegundos. Es coherente con el análisis global: las operaciones de pila siguen siendo $\Theta(1)$, pero el costo $\Theta(m)$ de editar el documento crece conforme este se alarga. Constituye un recordatorio de que el orden asintótico de una estructura no determina por sí solo el del programa que la emplea.

El arreglo supera a la lista con n grande (1560 frente a 1699 ms), pese a que ambas representaciones son $\Theta(1)$ por operación. La diferencia es de constantes, no de orden, y se explica por la jerarquía de memoria: el arreglo almacena los elementos de forma contigua y aprovecha las líneas de caché, mientras la lista dispersa los nodos por el montículo y paga fallos de caché con mayor frecuencia, además de una reserva dinámica por elemento.

Las mediciones en $n = 10^3$ no son concluyentes y así se reportan. La desviación estándar de la pila de arreglo (8.51) supera a su media (4.38). A ese tamaño el trabajo útil es tan breve que lo medido es principalmente el arranque del proceso: carga del ejecutable, primeros fallos de página y cacheo del archivo de entrada. Esto justifica la escala sugerida: solo a partir de 10^5 el tiempo de cómputo domina el ruido de medición.

Limitación metodológica. El tiempo reportado incluye la lectura y el análisis sintáctico del archivo de entrada, que es $O(n)$ y aporta una constante apreciable. Aislar el costo de las estructuras exigiría cargar los eventos en memoria antes de iniciar el cronómetro. Se documenta la limitación en lugar de omitirla, pues afecta la interpretación de las constantes aunque no la de los órdenes.

10. Conclusiones

Al empezar esta práctica pensaba que elegir entre una pila y una cola era una cuestión de preferencia. Trabajar los dos problemas me mostró lo contrario: la estructura la impone el patrón de acceso del problema.

En el editor, el orden de reversión es inverso al de ocurrencia y eso obliga a LIFO; en el firewall, los paquetes deben atenderse en su orden de llegada y eso obliga a FIFO. Cuando el patrón de acceso está claro, la estructura deja de ser una decisión y pasa a ser una consecuencia.

Lo que más me sorprendió fue comprobar que separar el tipo abstracto de su implementación no es un formalismo. Al escribir la lógica de los dos problemas contra una interfaz y no contra una clase concreta, pude cambiar la pila de arreglo por la de lista sin tocar una sola línea del editor. Que los siete casos de prueba produzcan salidas idénticas con ambas representaciones es la prueba de que el contrato se respeta, y me pareció la forma más convincente de entender qué significa realmente un TAD.

El análisis amortizado fue la parte que más me costó y la que más me enseñó. Terminé encontrándolo en tres lugares distintos: la duplicación de capacidad de la pila, el vaciado de la pila de rehacer y la purga de la ventana deslizante. En los tres el peor caso puntual resulta engañoso, porque una operación aislada puede costar $\Theta(n)$ mientras el costo real repartido sobre toda la ejecución es constante. Ver ese comportamiento confirmado en las mediciones —con el costo por paquete del firewall casi plano mientras n crecía tres órdenes de magnitud— fue lo que hizo que el argumento dejara de ser una fórmula y pasara a tener sentido.

Finalmente, los experimentos mostraron dos cosas que la notación asintótica oculta. La primera es que dos estructuras con el mismo orden pueden diferir en la práctica: el arreglo le ganó a la lista por localidad de memoria, no por complejidad. La segunda, y la que más me hizo pensar, es que el costo del dominio puede dominar al de la estructura: las operaciones de la pila son $\Theta(1)$, pero el Problema 1 crece de forma superlineal porque editar el documento cuesta $\Theta(m)$. Analizar la estructura no basta para analizar el programa que la usa.

11. Uso de herramientas de inteligencia artificial

Se utilizó Claude (Anthropic) como herramienta de apoyo durante el desarrollo del trabajo. Principalmente, se empleó para consultar alternativas de diseño para las estructuras implementadas, resolver dudas puntuales sobre algunas operaciones del código y recibir sugerencias generales sobre la organización del informe. Las decisiones finales de diseño, implementación, pruebas y validación del código fueron realizadas y verificadas de manera independiente. Asimismo, se utilizó como apoyo para aprender y manejar el formato L^AT_EX, debido a la falta de experiencia previa con esta herramienta, especialmente en la organización y maquetación del informe.

12. Referencias

1. Cormen, T. H., Leiserson, C. E., Rivest, R. L. y Stein, C. *Introduction to Algorithms*. 3.^a ed. Cambridge, MA: MIT Press, 2009. Capítulo 10, “Elementary Data Structures”, y capítulo 17, “Amortized Analysis”.
2. Weiss, M. A. *Data Structures and Algorithm Analysis in C++*. 4.^a ed. Boston: Pearson, 2014. Capítulo 3, “Lists, Stacks and Queues”.
3. cppreference.com. *Standard library header <chrono>*. Documentación de la biblioteca estándar de C++. Disponible en: <https://en.cppreference.com/w/cpp/header/chrono>

13. Contribución individual

Trabajo realizado de forma individual por **Nash Díaz**, quien diseñó las interfaces de los tipos abstractos, implementó las cuatro estructuras y la lógica de ambos problemas, construyó los catorce casos de prueba, ejecutó y analizó los experimentos, y redactó este informe.